Cdl: Informatica
Insegnamento: Programmazione e laboratorio 1

*Prof. Domenico Garlisi*
domenico.garlisi@unipa.it

# ESERCITAZIONE 3

## VETTORI

# Esercizio 16 - Simula lancio dadi

Scrivete un programma che simula il lancio di due dadi.

Il programma deve utilizzare la funzione rand() per lanciare il primo dado e successivamente il secondo.

```
#include <stdlib.h>
int face = 1 + rand() % 6; // random number from 1 to 6
```

Dopo il lancio dei dadi il programma deve calcolare la somma dei due valori.

Il programma deve lanciare i dadi per 36000 volte. Utilizzate un array unidimensionale per annotare tutte le occorrenze di ogni possibile somma e determinare il valore più probabile.

Nota: il valore più probabile dovrebbe essere il 7.

# Esercizio 17

Formalizzare l’algoritmo, per risolvere il seguente problema, quindi implementare la versione in codice c:

Una classe di dieci studenti ha svolto un quiz. I voti (interi nell'intervallo da 0 a 100) di questo quiz sono salvati in un array (opportunamente inizializzato con valori random).

Determina la media della classe nel quiz.

Inotre, il programma deve contare e visualizzare a video il numero di studenti che hanno superato la prova, sapendo che l’esame si intende superato con un voto maggiore o uguale a 50.

(La media della classe è la somma dei voti divisa per il numero di studenti.)

# Esercizio 18

Formalizzare l'algoritmo, per risolvere il seguente problema, quindi implementare la versione in codice c:

Siano dati in input un vettore $v_1$ di interi (di dimensione $n$, in input) ed un intero $k$. Si determini l'elemento di $v_1$ più prossimo a $k$, e lo si stampi assieme all'indice corrispondente.

# Esercizio 19

**Ricerca di un elemento**

Scrivere un programma in C che

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 99.

2) nel main, dato un valore X, inserito dall'utente, restituisca l'indice della prima occorrenza di X nell'array o -1 se non presente.

3) utilizzi una funzione che, dato un valore X, inserito dall'utente, restituisca l'indice della prima occorrenza di X nell'array o -1 se non presente.

# Esercizio 20

**Conteggio di occorrenze**

Scrivere un programma in C che

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 99.

2) nel main, dato un numero X, conti e restituisca il numero di volte in cui X compare nel vettore.

3) implementi una funzione che, dato un numero X, conti e restituisca il numero di volte in cui X compare nel vettore.

# Esercizio 21

**Ricerca del massimo e del minimo**

Scrivere un programma in C che

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 99.

2) nel main: calcola il valore massimo e il valore minimo contenuto nel vettore.

3) implementi due funzioni: una che restituisca il valore massimo e una che restituisca il valore minimo contenuto nel vettore.

# Esercizio 22 - Ricerca di pattern (sotto-array)

Scrivere un programma in C che

1) Generi un array principale di N interi e un array secondario di M interi, con M<N. (Scegliere arbitrariamente come inizializzare i vettori)

2) nel main verifichi se l'array secondario appare come sotto-sequenza consecutiva nell'array principale e restituisca l'indice di inizio o -1.

3) Implementi una funzione che verifichi se l'array secondario appare come sotto-sequenza consecutiva nell'array principale e restituisca l'indice di inizio o -1.

# Esercizio 23 - Rimozione di un elemento

Scrivere un programma in C

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 9.

2) nel main, dato un valore X inserito dall'utente, elimini tutte le occorrenza di X nel vettore spostando gli elementi successivi a sinistra e riempiendo gli elementi finali non più utilizzati con 0.

3) implementi una funzione che, dato un valore X inserito dall'utente, elimini tutte le occorrenza di X nel vettore spostando gli elementi successivi a sinistra e riempiendo gli elementi finali non più utilizzati con 0.

# Esercizio 24 - Duplicati

Scrivere un programma in C

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 9.

2) nel main crei un nuovo array contenente solo i valori unici (senza duplicati) e stampi il risultato.

3) implementi una funzione che crei un nuovo array contenente solo i valori unici (senza duplicati) e stampi il risultato.

# Esercizio 25 - Shift circolare

Scrivere un programma in C

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 9.

2) nel main esegua uno shift circolare a destra di K posizioni (gli ultimi elementi tornano in testa), con k inserito dall'utente e stampi il vettore risultante.

3) implementi una funzione che esegua uno shift circolare a destra di K posizioni (gli ultimi elementi tornano in testa), con k inserito dall'utente e stampi il vettore risultante.

# Esercizio 26 - Suddivisione in pari e dispari

Scrivere un programma in C

1) inizializzi un array di N elementi interi (con N inserito dall'utente) con valori random compresi tra 0 e 9.

2) nel main produca:

un nuovo vettore contenente solo i numeri pari,

un nuovo vettore contenente solo i numeri dispari.

3) implementi due funzioni che producano:

un nuovo vettore contenente solo i numeri pari,

un nuovo vettore contenente solo i numeri dispari.

# Esercizio 27

Building a Casino Game (funzioni) - Un gioco di dadi popolare noto come "craps".

- Un giocatore lancia due dadi.
- Ogni dado ha sei facce. Queste facce contengono 1, 2, 3, 4, 5 e 6 punti.
- Dopo che i dadi si sono fermati, si calcola la somma dei punti sulle due facce rivolte verso l'alto.
- *Se la somma è 7 o 11 al primo tiro, il giocatore vince.*
- *Se la somma è 2, 3 o 12 al primo tiro (chiamato "craps"), il giocatore perde (cioè, "la casa" vince).*
- *Se la somma è 4, 5, 6, 8, 9 o 10 al primo tiro, quella somma diventa il "punto" del giocatore.*
- Per vincere, il giocatore deve continuare a lanciare i dadi fino a quando non "fai il punto".
- Il giocatore perde trovando un 7 prima di fare il punto.

- All'inizio del gioco l'utente deve inserire il suo identificativo tra 0 e 2. Utilizzare il questo identificativo come indice dell'elemento dei due vettori in cui memorizzare le info dei giocatori.
- Bisogna tenere traccia (salvare) i punteggi di ogni giocatore e il numero di partite eseguite.
- I punteggi di ogni giocatore (classifica) e il numero di partite devono essere salvati e mostrati alla fine di ogni partita.
- Il gioco si chiude se l'utente inserisce -1, o se un giocatore raggiunge 3 punti.
- Utilizzare una funzioni separate per l'esecuzione di una partita, la stampa dei riassunti delle partite.

# Modalità di implementazione

1. Si crei una funzione per implementare il lancio del dado;
2. Si crei una funzione per implementare la partita di un giocatore;
3. Si utilizzi la funzione Rand per simulare il lancio del dado;
4. Si crei una funzione per stampare la classifica;
5. Si utilizzi un ciclo do .. while per eseguire multiple chiamate del lancio del dado;
6. Si utilizzi il costrutto Switch - case per valutare l'esito del lancio dei dadi;
7. Si utilizzi un ciclo do .. while per eseguire partite multiple;